Biogeochemical Modelling of Lakes, Sediments and Oceans
701-1338-00L 2026

Numerical Modeling of Partial Differential Equations (PDEs)

Nathan Beech, Environmental Physics, ETHZ
nathan.beech@usys.ethz.ch

Slides adapted from: Abolfazl Irani Rahaghi, EAWAG, 2025

Download

Examples and Exercises:

Numerical_methods_Part_II_with_exercises.ipynb

and

Numerical_methods_Part_II_with_exercises.py

Solution:

Numerical_methods_Part_II_with_solutions.ipynb

and

Numerical_methods_Part_II_with_solutions.py

Review: functions and derivatives

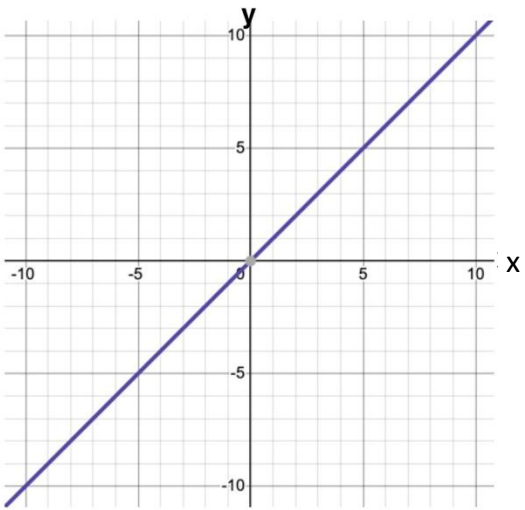
Function: An equation describing the relationship of one variable to another

Note: the variable y is often written as f(x)

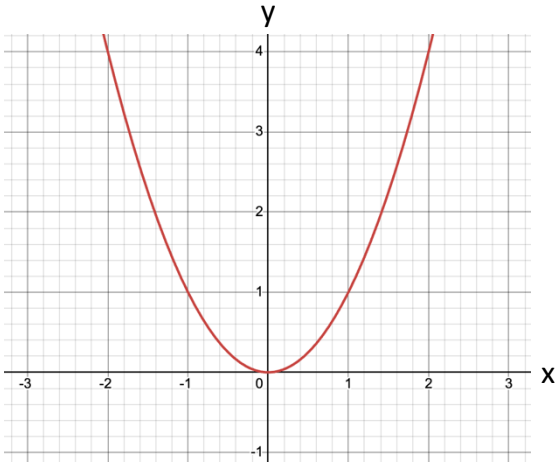
- Temperature at a given time: T(t)
- Pressure at a given depth: P(z)

Examples:

Linear: $y = x$



Parabolic: $y = x^2$

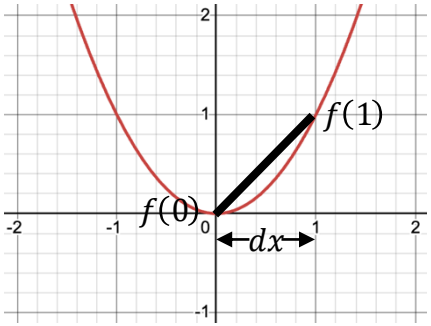


Review: functions and derivatives

Derivative: An equation describing the slope of a function – the rate of change of a variable in relation to another

Derivatives can be approximated from specific points on the function

$$f'(0.5) = \frac{f(1) - f(0)}{dx}$$



Note:

- The derivative of the function f(x) is written f'(x) or df/dx (our other examples: dT/dt, dP/dz)
- The second order derivative (derivative of the derivative) is written f''(x) or $\mathrm{d}^2\mathrm{f}/\mathrm{d}x^2$

Derivatives can be derived from the function mathematically

$$f(x) = x^2$$
$$f'(x) = 2x$$

Second order derivative example: distance at a given time

- d(t) describes your distance, relative to a starting point, at a given time: your position (in metres from the starting point)
- d'(t) describes the rate of change of your position at a given time: your velocity (in m/s)
- d''(t) describes the rate of change of your velocity at a given time: your acceleration (in m/s^2)

Review: differential equations

	function	differential equation
Differential equation: An equation relating a function to its derivatives	$u(t)$	$du/dt = \alpha\, u(t)$
Partial differential equation: An equation relating a multi-dimensional function to its derivatives in one dimension	$u(x,t)$	partial differential equation $\partial u/\partial t = \alpha\, \partial^2 u/\partial x^2$

Note: the partial derivative is "partial" because it describes change in only one dimension, while the variable u changes in two dimensions – it partially defines the rate of change

ODE numerical modeling - Recap

- Step 0: Problem formulation $$\frac{dC(t)}{dt} = -kC(t)$$
- Step 1: Domain discretization

$t_n = n\Delta t$ with $n = 0, 1, .., N$ and $\Delta t = T/N$

- Step 2: Fulfilling the eq. at discrete grid points $$\frac{dC^n}{dt} = -kC^n$$
- Step 3: Replacing derivatives by finite difference

$$\frac{dC^n}{dt} = \frac{C^{n+1} - C^n}{\Delta t}$$

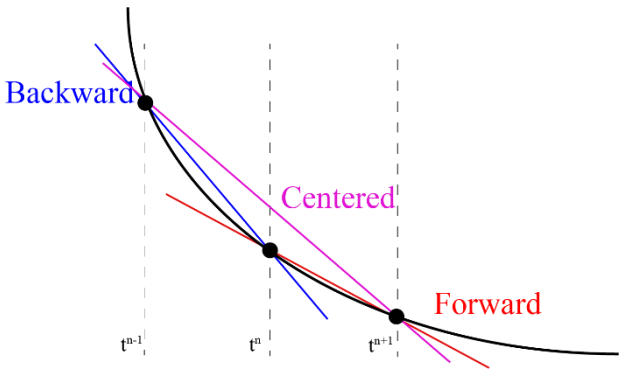
- Step 4: Formulating a recursive algorithm

$$\frac{C^{n+1} - C^n}{\Delta t} = -kC^n \quad \Longrightarrow \quad C^{n+1} = (1 - k\Delta t)C^n$$

- Step 5: Implementing the numerical script and visualizing the results

ODE numerical modeling - Recap…

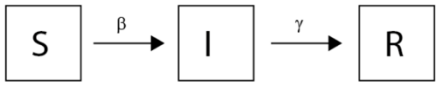
- Different discretization of the first order derivative



$$C^{n+1} = \frac{1-(1-\theta)k\Delta t}{1+\theta k\Delta t}C^{n}$$

- Verification of the numerical model (Step 6): stability, consistency, and convergence

- Numerical modeling of a system of ODEs



$$\frac{\partial S}{\partial t} = -\beta I(t)S(t)$$

$$\frac{\partial I}{\partial t} = \beta I(t)S(t) - \gamma I(t)$$

$$\frac{\partial R}{\partial t} = \gamma I(t)$$

Partial Differential Equations

- Include various **partial derivatives** of a **multivariable function**, for example:

1D diffusion equation: $$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial z^2}$$

1D advection equation: $$\frac{\partial u}{\partial t} + v \frac{\partial u}{\partial x} = 0$$

Laplace's equation: $$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} = 0$$

1D Wave equation: $$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

pde package: Reminder from the 1st and 2nd lectures

Differential equation

```python
# PDE function
def simple_pde(z, C, t, dC_dz):
    # Define the vertical diffusivity
    Kz = 1e-1  # m2 / d

    # Define first order reaction rate
    kr = 1e-2  # [d - 1]

    # Define area function
    A = 200000 - 10000*z #trapezoid lake; area in m2

    # Define the PDE
    # A * dC/dt = Kz * A * d2C/dz2 - kr * A * C
    c = [A]
    f = [Kz * A * dC_dz]
    s = [-kr * A * C]

    return [c, f, s]
```

pde_solver (from pde import *)

```python
# Call pde_solver. It needs the following arguments:
# simple_pde: function containing the PDEs
# BC_left: BoundaryCondition containing conditions for z = 0
# BC_right: BoundaryCondition containing conditions for z = Zmax
# simple_IC: function containing initial condition for t=0 at all z
# z: z grid
# t: t grid
sol = pde_solver(simple_pde, BC_left, BC_right, simple_IC, z, t)
```

Boundary conditions

```python
# Boundary conditions:
# On the left, a dirichlet boundary condition C(t) = 2
# On the right, a neumann boundary condition dC/dz = 0
def neumann_function(t, u):
    return 0


# You need to have f=function for neumann conditions!
BC_left = BoundaryCondition(1, p=2)
BC_right = BoundaryCondition(2, f=neumann_function)
```

Time and space discretization

```python
Zmax = 10  # Lake depth [m]
tmax = 200  # maximum simulation time [d]

M = 2001  # number of timesteps
N = 201  # number of spacial nodes

t = np.linspace(0, tmax, M)  # time discretization
z = np.linspace(0, Zmax, N)  # space discretization
```

Initial conditions

```python
# Initial conditions
def simple_IC(x):
    # Initial conditions for t = 0
    # Here we set C0 = 0
    return [0]
```

pde package …

```
# Solve PDE problem using solve_ivp
solution = solve_ivp(lambda t, u: pde_fun(t, u, x, dims, bcs_left, bcs_right, pde, dx, dx_2, counter, index),
                     y0=initial, t_span=[t[0], t[-1]], t_eval=t, method='BDF')
```

BDF: Backward Differentiation Formula

```
from itertools import count
from inspect import getfullargspec
import numpy as np
from numba import jit
from scipy.integrate import solve_ivp
import warnings
from numba.core.errors import NumbaWarning, NumbaDeprecationWarning
```

method : ***string or*** `OdeSolver`***, optional***

Integration method to use:

- 'RK45' (default): Explicit Runge-Kutta method of order 5(4) [1]. The error is controlled assuming accuracy of the fourth-order method, but steps are taken using the fifth-order accurate formula (local extrapolation is done). A quartic interpolation polynomial is used for the dense output [2]. Can be applied in the complex domain.
- 'RK23': Explicit Runge-Kutta method of order 3(2) [3]. The error is controlled assuming accuracy of the second-order method, but steps are taken using the third-order accurate formula (local extrapolation is done). A cubic Hermite polynomial is used for the dense output. Can be applied in the complex domain.
- 'DOP853': Explicit Runge-Kutta method of order 8 [13]. Python implementation of the "DOP853" algorithm originally written in Fortran [14]. A 7-th order interpolation polynomial accurate to 7-th order is used for the dense output. Can be applied in the complex domain.
- 'Radau': Implicit Runge-Kutta method of the Radau IIA family of order 5 [4]. The error is controlled with a third-order accurate embedded formula. A cubic polynomial which satisfies the collocation conditions is used for the dense output.
- 'BDF': Implicit multi-step variable-order (1 to 5) method based on a backward differentiation formula for the derivative approximation [5]. The implementation follows the one described in [6]. A quasi-constant step scheme is used and accuracy is enhanced using the NDF modification. Can be applied in the complex domain.
- 'LSODA': Adams/BDF method with automatic stiffness detection and switching [7], [8]. This is a wrapper of the Fortran solver from ODEPACK.

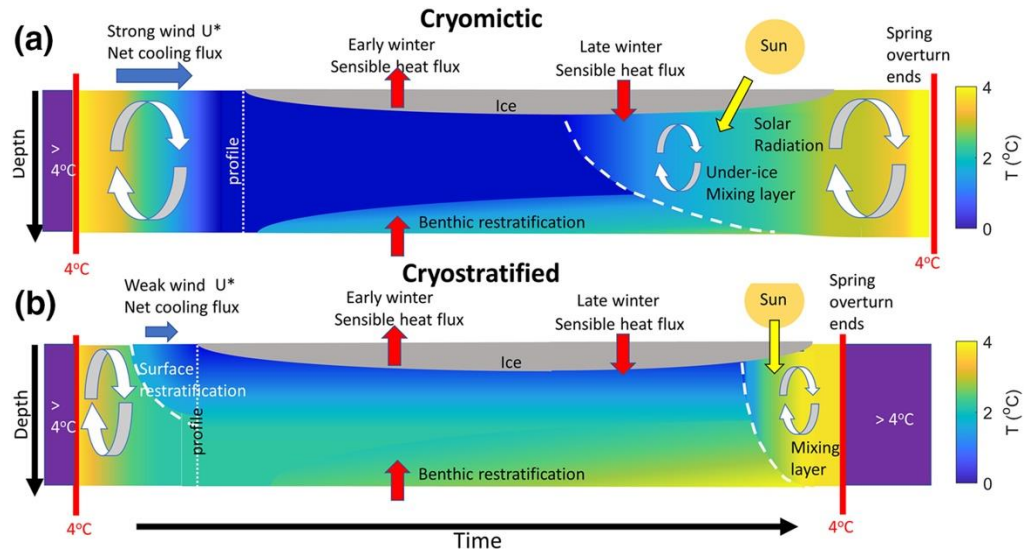
[5] https://en.wikipedia.org/wiki/Backward_differentiation_formula

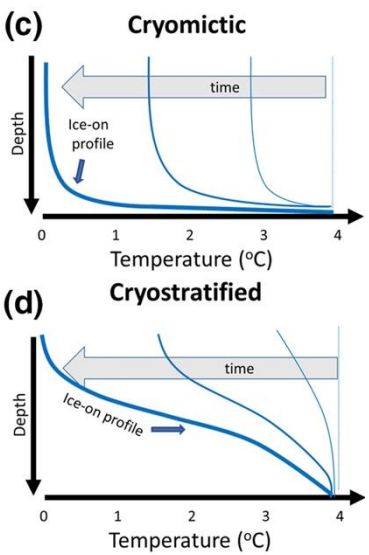
[6] L. F. Shampine, M. W. Reichelt, "THE MATLAB ODE SUITE". 10.1137/S1064827594276424

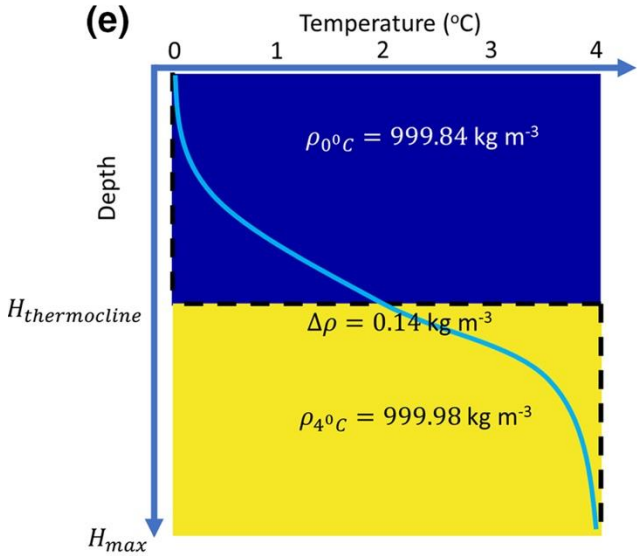
Objectives

- Numerical modeling of simple PDEs using finite difference technique
  - How to discretize the derivatives?
  - How to formulate a recursive algorithm?
  - Implement the algorithm in Python
  - Verify the numerical model

Temperature of ice-covered lakes







Goal: to evaluate the evolution of the thermal structure in an ice-covered lake (depth H = 10 m) over one month

Approach: Finite difference discretization, i.e, from a 2-box model to N-box model

Assumptions:

- Lake fully insolated (no heat exchange)
- Temperature at the air-ice interface: 0°C (constant)
- Temperature at the lake bottom: 4°C (constant)

* **Drichilet boundary conditions:** fixed values at boundaries

Source: Yang et al. 2020. https://doi.org/10.1029/2020GL091374

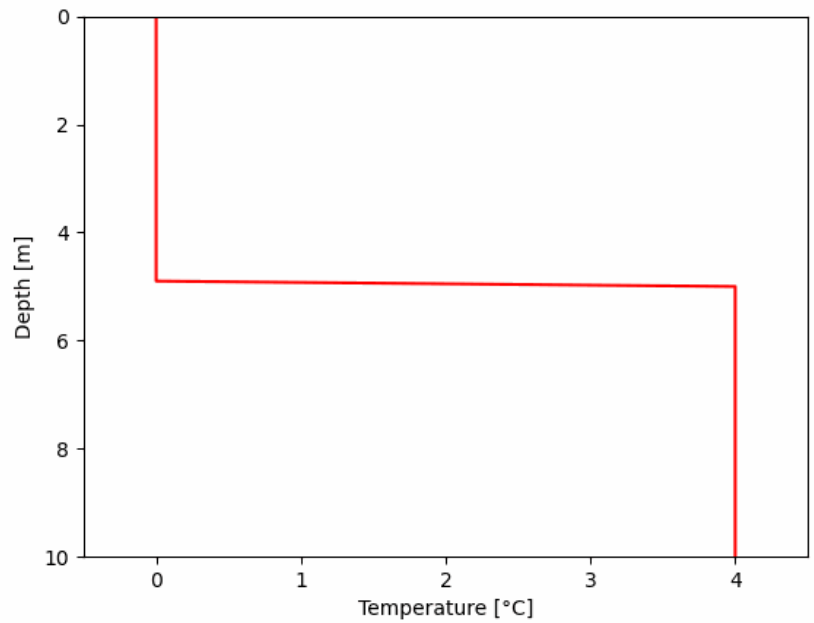
Initial-boundary value problem

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial z^2}, \qquad z \in [0, H],\ t \in [0, T]$$

Also known as **heat equation**

- $u(z,t)$ is the unknown function to be solved (here temperature [°C]) both in space (along z coordinate) and time (along t coordinate)
- α is the diffusion coefficient [m^2s^{-1}]
- To solve the equation, we need **initial** and **boundary** conditions. Therefore, the 1-D diffusion equation is an initial-boundary value problem.

Example: Lake temperature begins stratified, and equilibrates over time



Numerical modeling of 1-D heat equation

- Step 0: Problem formulation

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial z^2}, \quad z \in [0, H],\ t \in [0, T]$$

- First-order derivative in time: one initial condition needed in time, $u(z,0)=T(z)$, $z\in[0,H]$, e.g.,

$$T(z,0) = \begin{cases} T_0 = 0, & 0 < z < \frac{H}{2}, \\ T_H = 4, & \frac{H}{2} \le z \le H, \end{cases}$$

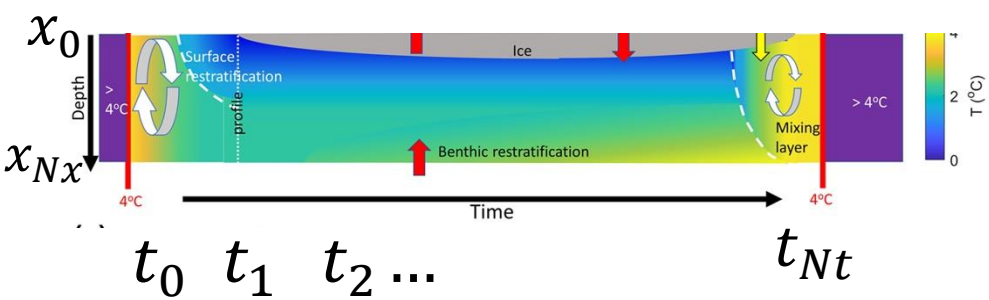
- Second-order derivative in space: two boundary conditions needed in space, e.g.,

$$\begin{cases} u(0,t) = T_0, & t > 0, \\ u(H,t) = T_H, & t > 0\,. \end{cases}$$

Numerical modeling of 1-D heat equation…

- Step 1: Domain discretization

$$\begin{cases} x_i = i\Delta x, & i = 0, \dots, N_x, \\ t_n = n\Delta t, & n = 0, \dots, N_t\,. \end{cases}$$

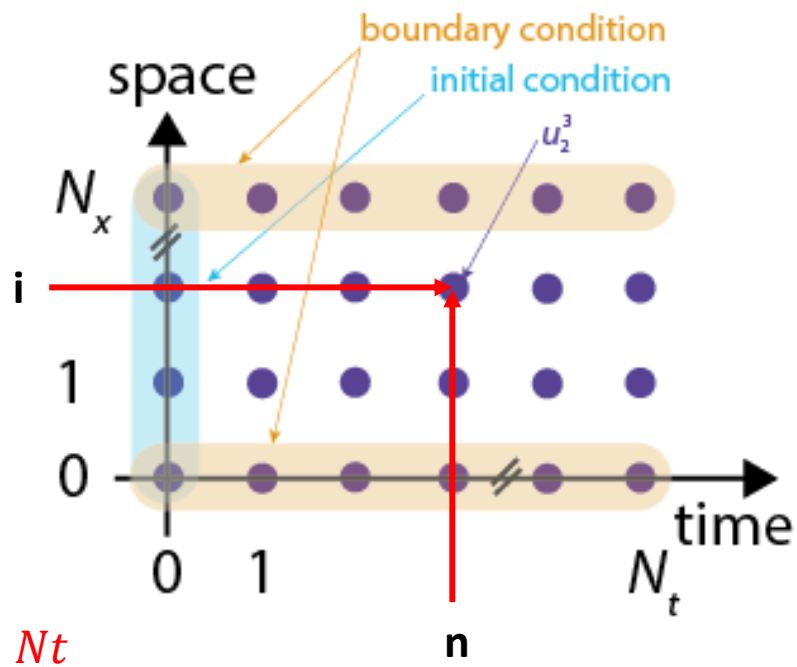


- Step 2: Fulfilling the eq. at discrete grid points

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

$$\frac{\partial}{\partial t} u(x_i, t_n) = \alpha \frac{\partial^2}{\partial x^2} u(x_i, t_n),$$

$$\frac{\partial u_i^n}{\partial t} = \alpha \frac{\partial^2 u_i^n}{\partial x^2},$$

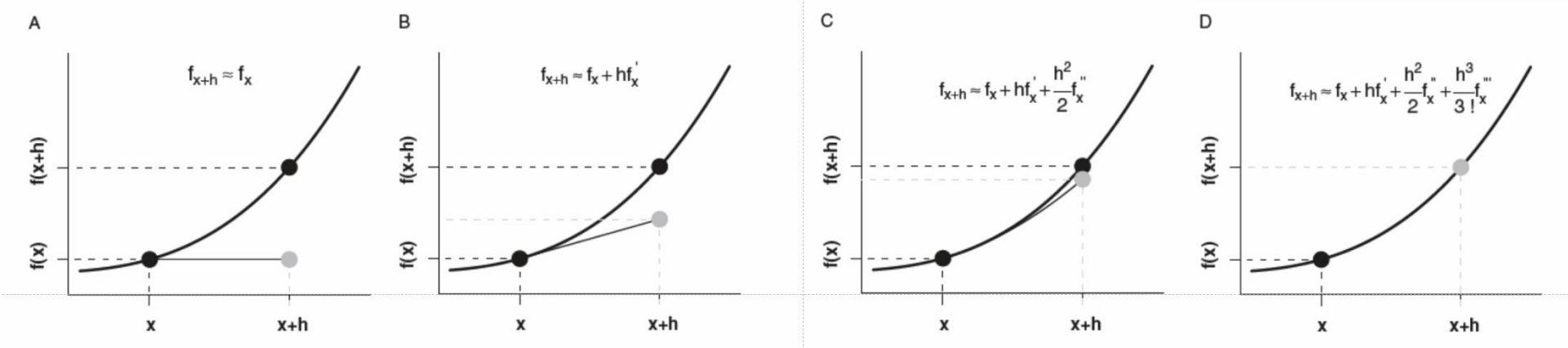


- upper indice n for time with $n = 0, \dots, Nt$
- lower indice i for space with $i = 0, \dots, Nx$

- Step 3: Replacing derivatives by finite difference

Taylor series decomposition reminder:

$$f(t_n + \Delta t) = f(t_n) + \frac{\Delta t}{1!}\frac{df}{dt} + \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} + \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} + \ldots + + \frac{(\Delta t)^n}{n!}\frac{d^n f}{dt^n}$$



Taylor series must be truncated, e.g.,:

$$f(t_n + \Delta t) = f(t_n) + \frac{\Delta t}{1!}\frac{df}{dt} + \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} + \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} + O((\Delta t)^4))$$

- Step 3: Replacing derivatives by finite difference

first-order derivative (time):

$$f(t_n + \Delta t) = f(t_n) + \frac{\Delta t}{1!}\frac{df}{dt} + \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} + \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} + O((\Delta t)^4))$$

$$\Delta t \frac{df}{dt} = f(t_n + \Delta t) - f(t_n) - \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} - \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} - O((\Delta t)^4))$$

$$\frac{df}{dt} = \frac{f(t_n + \Delta t) - f(t_n)}{\Delta t} - \frac{\Delta t}{2!}\frac{d^2 f}{dt^2} - \frac{(\Delta t)^2}{3!}\frac{d^3 f}{dt^3} - O((\Delta t)^3))$$

Forward Euler or Forward difference approximation

$$\frac{df}{dt} = \frac{f(t_n + \Delta t) - f(t_n)}{\Delta t} - O(\Delta t)$$

- Step 3: Replacing derivatives by finite difference

first-order derivative (space):

(1) $$f(x_i + \Delta x) = f(x_i) + \frac{\Delta x}{1!}\frac{df}{dx} + \frac{(\Delta x)^2}{2!}\frac{d^2 f}{dx^2} + \frac{(\Delta x)^3}{3!}\frac{d^3 f}{dx^3} + O((\Delta x)^4)$$ Forward Taylor expansion

(2) $$f(x_i - \Delta x) = f(x_i) - \frac{\Delta x}{1!}\frac{df}{dx} + \frac{(\Delta x)^2}{2!}\frac{d^2 f}{dx^2} - \frac{(\Delta x)^3}{3!}\frac{d^3 f}{dx^3} + O((\Delta x)^4)$$ Backward Taylor expansion

(1) – (2) $$f(x_i + \Delta x) - f(x_i - \Delta x) = 2\Delta x \frac{df}{dx} + O((\Delta x)^3)$$

$$\frac{df}{dx} = \frac{f(x_i + \Delta x) - f(x_i - \Delta x)}{2\Delta x} + O((\Delta x)^2)$$ Centered difference method

second-order derivative (space):

(1) + (2) $$f(x_i + \Delta x) + f(x_i - \Delta x) = 2f(x_i) + (\Delta x)^2 \frac{d^2 f}{dx^2} + O((\Delta x)^4)$$

$$\frac{d^2 f}{dx^2} = \frac{f(x_i + \Delta x) - 2f(x_i) + f(x_i - \Delta x)}{(\Delta x)^2} + O((\Delta x)^2)$$

- Step 3: Replacing derivatives by finite difference

Forward Euler method or explicit method:

(from slide 16)

$$\frac{df}{dt} = \frac{f(t_n + \Delta t) - f(t_n)}{\Delta t} - O(\Delta t)$$

(from slide 17)

$$\frac{d^2 f}{dx^2} = \frac{f(x_i + \Delta x) - 2f(x_i) + f(x_i - \Delta x)}{(\Delta x)^2} + O((\Delta x)^2)$$

From f(t) and f(x) to f(x,t)

$$\frac{\partial u}{\partial t} = \frac{u(t_n + \Delta t) - u(t_n)}{\Delta t} - O(\Delta t)$$

$$\frac{\partial^2 u}{\partial x^2} = \frac{u(x_i + \Delta x) - 2u(x_i) + u(x_i - \Delta x)}{(\Delta x)^2} + O((\Delta x)^2)$$

Back to the governing equation:
(1D diffusion equation/Heat equation from our lake example)

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

$$\frac{u(t_n + \Delta t) - u(t_n)}{\Delta t} - O(\Delta t) = \alpha \frac{u(x_i + \Delta x) - 2u(x_i) + u(x_i - \Delta x)}{(\Delta x)^2} + O((\Delta x)^2)$$

Using our notation for discretized grid points

$$\boxed{\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}}$$

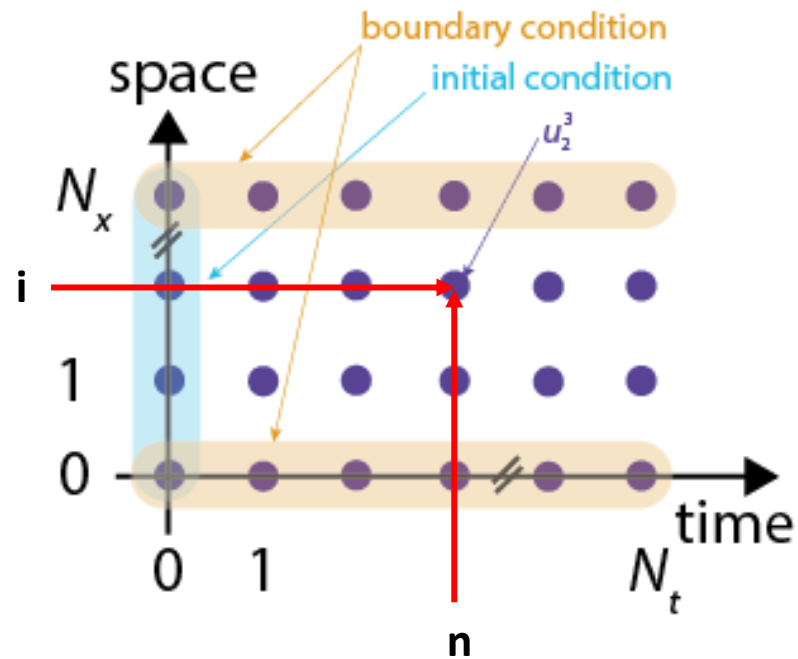
- Step 4: Formulating a recursive algorithm

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}.$$

$$u_i^{n+1} = u_i^n + \frac{\alpha \Delta t}{\Delta x^2}(u_{i+1}^n - 2u_i^n + u_{i-1}^n).$$

$$u_i^{n+1} = u_i^n + F(u_{i+1}^n - 2u_i^n + u_{i-1}^n).$$

with $F = \frac{\alpha \Delta t}{\Delta x^2}$ a nondimensional number.



Computational algorithm:

1. Compute $u_i^0 = I(x_i)$ for i=0..N_x
2. for n=0,1,..,N_t
   - run the equation $u_i^{n+1} = u_i^n + F(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$ for all the internal spatial points i=1,...,N_{x-1}
   - set the boundary values $u_i^{n+1} = 0$ for i=0 and $u_i^{n+1} = 4$ for i=N_x

- Step 5: Implementing the numerical script and visualizing the results

Coding Example 1 & Exercise 1

- Write a script to implement a numerical scheme following the computational algorithm
- Plot the results

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

Exercise 2: Testing the code in Python (Step 6)

Coding Exercise 2

- Try to run the code by changing diffusivity (maybe increase diffusivity to 10^{-5} or 10^{-4} m^2s^{-1})
- Decrease dx or dt. Each time output the parameter F
  - Case 1: keep dx= 1 m and decrease dt. for instance: 1h , 30 min, 1 min, 1 s
  - Case 2: keep dt= 1 h and decrease dx. for instance: 1 m , 0.5 m, 0.1 m
- Draw conclusions regarding stability

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

One can show that the forward method for diffusion equation is conditionally stable. ⇒ $F \leq \frac{1}{2}$ or $\Delta t \leq \frac{\Delta x^2}{2D}$

See the **"Supplemental Information"** slides below!

- Back to Step 3: Replacing derivatives by finite difference

Assuming Dirichlet boundary conditions:

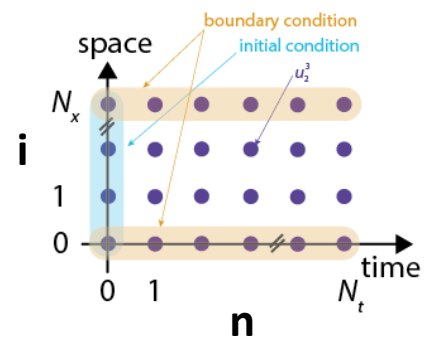
$$u(0,t) = T_0, \quad t > 0,$$
$$u(H,t) = T_H, \quad t > 0\,.$$

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

$$\frac{u_i^n - u_i^{n-1}}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

$$-Fu_{i-1}^n + (1+2F)u_i^n - Fu_{i+1}^n = u_i^{n-1}$$

Recall:



$i = 1$ $\quad -Fu_0^n + (1+2F)u_1^n - Fu_2^n = u_1^{n-1}$

$i = 2$ $\quad -Fu_1^n + (1+2F)u_2^n - Fu_3^n = u_2^{n-1}$

$i = 3$ $\quad -Fu_2^n + (1+2F)u_3^n - Fu_4^n = u_3^{n-1}$

$\vdots$

$i = N_x - 1$ $\quad -Fu_{N_{x-2}}^n + (1+2F)u_{N_{x-1}}^n - Fu_{Nx}^n = u_{Nx-1}^{n-1}$

Backward Euler method or Implicit method…

- Back to Step 3: Replacing derivatives by finite difference

A tri-diagonal system to solve:

$$\begin{bmatrix} 1+2F & -F & 0 & \ddots & \ddots & \ddots & \vdots \\ -F & 1+2F & -F & 0 & \ddots & \ddots & \vdots \\ 0 & -F & 1+2F & -F & 0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & \cdots & \cdots & -F & 1+2F & -F \\ 0 & \cdots & \cdots & \cdots & \cdots & -F & 1+2F \end{bmatrix} \begin{bmatrix} u_1^n \\ u_2^n \\ \vdots \\ \vdots \\ \vdots \\ u_{Nx-2}^n \\ u_{Nx-1}^n \end{bmatrix} = \begin{bmatrix} u_1^{n-1} \\ u_2^{n-1} \\ \vdots \\ \vdots \\ \vdots \\ u_{Nx-2}^{n-1} \\ u_{Nx-1}^{n-1} \end{bmatrix} + \begin{bmatrix} Fu_0^n \\ 0 \\ \vdots \\ \vdots \\ \vdots \\ 0 \\ Fu_{Nx}^n \end{bmatrix}$$

$$AU^{n+1} = U^n + B$$

Boundary conditions

Backward Euler method or Implicit method…

- Back to Step 4: Formulating a recursive algorithm

$$\frac{u_i^n - u_i^{n-1}}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

$$-Fu_{i-1}^n + (1+2F)u_i^n - Fu_{i+1}^n = u_i^{n-1}$$

$$AU^{n+1} = U^n + B$$

$$\boxed{U^{n+1} = A^{-1}(U^n + B)}$$

Computational algorithm:

1. Compute $u_i^0 = I(x_i)$ for i=0..N_x
2. for n=0,1,..,N_t Solve the matrix $U^{n+1} = A^{-1}(U^n + B)$

Example 2: Implementing backward method code in python

- Write a script to implement a numerical scheme following the computational algorithm in previous slide
- Plot the results
- Try to run the code by changing diffusivity (maybe increase diffusivity to 10^{-5} or 10^{-4} m^2s^{-1})
- Decrease dx or dt. Each time output the parameter F
  - Case 1: keep dx= 1 m and decrease dt. for instance: 1h , 30 min, 1 min, 1 s
  - Case 2: keep dt= 1 h and decrease dx. for instance: 1 m , 0.5 m, 0.1 m
- Draw conclusions regarding stability

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

One can show that the backward method for diffusion equation is unconditionally stable.

See the **"Supplemental Information"** slides below!

Numerical modeling with Neumann boundary condition

Goal: to evaluate the evolution of the thermal structure in an ice-covered lake (depth H = 10 m) over one month

Hypothesis:

- Temperature at the air-ice interface: 0°C (constant)
- Sediment/geothermal flux at the bottom: $\frac{\partial T}{\partial z}|_{z=bottom} = 0.1\ °C\ m^{-1}$

- Back to Step 3: Replacing derivatives by finite difference

Assuming Neumann boundary conditions: $u(0,t) = T_0, \quad t > 0, \frac{\partial u(H,t)}{\partial x} = \lambda, \quad t > 0$

Using backward Euler or implicit method:

$$\frac{u_i^n - u_i^{n-1}}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

$$-Fu_{i-1}^n + (1+2F)u_i^n - Fu_{i+1}^n = u_i^{n-1}$$

Using a Leap-frog approximation:

$$\frac{\partial u}{\partial x} = \frac{u_{i+1}^n - u_{i-1}^n}{2\Delta x}$$

$$\frac{\partial u}{\partial x}|_{z=Nz} = \lambda$$

$$u_{Nx+1}^n = u_{Nx-1}^n - 2\lambda\Delta x$$

- Back to Step 3: Replacing derivatives by finite difference

$i = 1$ $\quad -Fu_0^n + (1+2F)u_1^n - Fu_2^n = u_1^{n-1}$

$i = 1$ $\quad u_1^n + (1+2F)u_1^n - Fu_2^n = u_1^{n-1}$

$i = 2$ $\quad -Fu_1^n + (1+2F)u_2^n - Fu_3^n = u_2^{n-1}$

$i = 3$ $\quad -Fu_2^n + (1+2F)u_3^n - Fu_4^n = u_3^{n-1}$

…

Substitute $\boxed{u_{Nx+1}^n = u_{Nx-1}^n - 2\lambda\Delta x}$

$i = N_x$ $\quad -Fu_{N_{x-1}}^n + (1+2F)u_{N_x}^n - Fu_{N_{x+1}}^n = u_{Nx}^{n-1}$

$i = N_x$ $\quad -2Fu_{N_{x-1}}^n + (1+2F)u_{N_x}^n = u_{Nx}^{n-1} + 2F\lambda\Delta x$

$$\begin{bmatrix} 1+2F & -F & 0 & \ddots & \ddots & \ddots & \vdots \\ -F & 1+2F & -F & 0 & \ddots & \ddots & \vdots \\ 0 & -F & 1+2F & -F & 0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & \cdots & \cdots & -F & 1+2F & -F \\ 0 & \cdots & \cdots & \cdots & \cdots & -2F & 1+2F \end{bmatrix} \begin{bmatrix} u_1^n \\ u_2^n \\ \vdots \\ \vdots \\ \vdots \\ u_{Nx-2}^n \\ u_{Nx-1}^n \end{bmatrix} = \begin{bmatrix} u_1^{n-1} \\ u_2^{n-1} \\ \vdots \\ \vdots \\ \vdots \\ u_{Nx-2}^{n-1} \\ u_{Nx-1}^{n-1} \end{bmatrix} + \begin{bmatrix} Fu_0^n \\ 0 \\ \vdots \\ \vdots \\ \vdots \\ 0 \\ 2F\lambda\Delta x \end{bmatrix}$$

Example 3: Implementing the code using Neumann boundary condition

- Revise the backward Euler script assuming Dirichlet boundary condition at the surface and Neumann boundary condition at deeper part
- Plot the results

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

Find more examples in the supplement that you can try in Jupyter

- 1D advection equation
- Ex. 4 Forward in time centered in space (FTCS) method for the 1D advection equation
- Ex. 5 Lax Frtiedrich's scheme for the 1D advection equation

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

Biogeochemical Modelling of Lakes, Sediments and Oceans
701-1338-00L 2026

Numerical Modeling of Partial Differential Equations (PDEs)

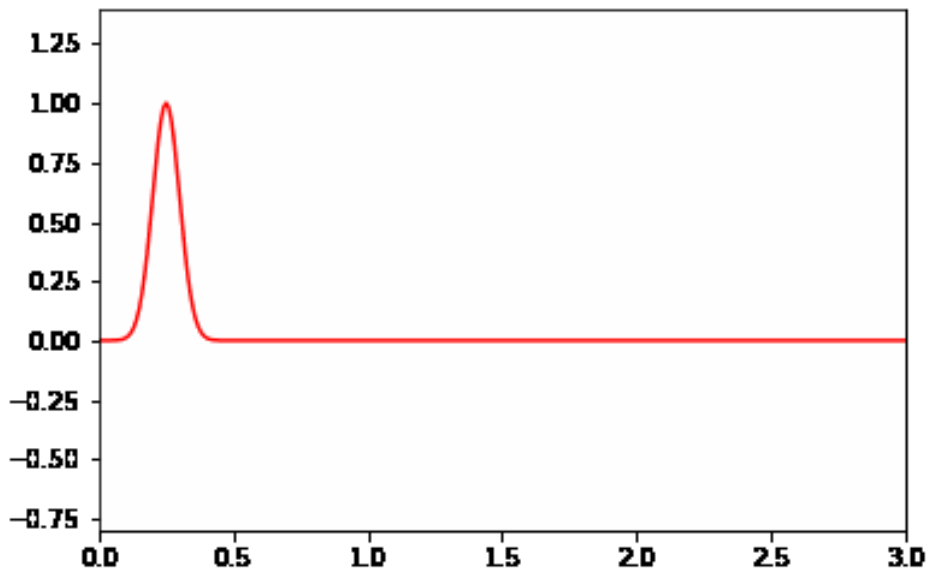
Supplemental Information

Slides from Abolfazl Irani Rahaghi, abolfazl.irani@eawag.ch

Part 2 – 1D advection equation

- Scalar advection equation (advection of some scalar field $u(x,t)$ carried along by a flow of constant speed v) takes the form:

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = 0$$



Numerical modeling of 1-D advection equation…

- Step 1: Domain discretization

$$\begin{cases} x_i = i\Delta x, & i = 0, \dots, N_x, \\ t_n = n\Delta t, & n = 0, \dots, N_t\,. \end{cases}$$

- Step 2: Fulfilling the eq. at discrete grid points

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = 0$$

$$\frac{\partial u_i^n}{\partial t} = -v\frac{\partial u_i^n}{\partial x},$$

  - upper indice n for time with n = 0, …, Nt
  - lower indice i for space with i = 0, …, Nx

- Step 3: Replacing derivatives by finite difference

Forward in Time - Centered in Space (FTCS):

$$\frac{\partial u}{\partial t} = -v\frac{\partial u}{\partial x}$$

$$\frac{\partial u}{\partial t} = \frac{u(t_n + \Delta t) - u(t_n)}{\Delta t}$$

$$\frac{\partial u}{\partial x} = \frac{u(x_i + \Delta x) - u(x_i - \Delta x)}{2\Delta x}$$

$$\boxed{u_i^{n+1} - u_i^n = -\frac{v\Delta t}{2\Delta x}\left(u_{i+1}^n - u_{i-1}^n\right)}$$

- Step 4: Formulating a recursive algorithm

$$u_i^{n+1} - u_i^n = -\frac{v\Delta t}{2\Delta x}\left(u_{i+1}^n - u_{i-1}^n\right)$$

Computational algorithm:

1. Compute $u_i^0 = I(x_i)$ for i=0..N_x
2. for n=0,1,..,N_t run the equation $u_i^{n+1} - u_i^n = -0.5C\left(u_{i+1}^n - u_{i-1}^n\right)$ for all the internal spatial points i=1,...,N_{x-1}

C is called **CFL** (Courant–Friedrichs–Lewy's) or simply **Courant's** number.

Example 4: Implementing and testing FTCS method in python

- Write a script to implement a numerical FTCS scheme to solve 1D advection equation
- Plot the results
- Test the code stability by using different *dt* and/or *dx*

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

One can show that the FTCS method for advection equation is unconditionally unstable.

See the **"Supplemental Information"** slides below!

Lax-Friedrichs method for solving 1D advection equation

- Back to Step 3: Replacing derivatives by finite difference

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = -\frac{v}{2\Delta x}(u_{i+1}^n - u_{i-1}^n)$$

which can be rearranged as:

$$u_i^{n+1} = u_i^n - \frac{v\Delta t}{2\Delta x}(u_{i+1}^n - u_{i-1}^n)$$

Let's replace u_i^n by: $u_i^n = \frac{1}{2}(u_{i+1}^n + u_{i-1}^n)$

- Back to Step 4: Formulating a recursive algorithm

$$\boxed{u_i^{n+1} = \frac{1}{2}(u_{i+1}^n + u_{i-1}^n) - \frac{v\Delta t}{2\Delta x}(u_{i+1}^n - u_{i-1}^n)}$$

Example 5: Implementing and testing Lax-Friedrichs method in python

- Revise the script to implement the numerical Lax-Friedrich scheme to solve 1D advection equation
- Plot the results
- Test the code stability by using different *dt* and/or *dx*

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

One can show that the Lax–Friedrichs method for advection equation is conditionally stable for $C \leq 1$. However, this method introduces artificial (numerical) diffusion to stabilize the advection equation

See the **"Supplemental Information"** slides below!

Lax-Wendroff method for solving 1D advection equation

- Back to Step 3 and 4

2nd order accuracy in space and time:

$$\boxed{u_i^{n+1} = u_i^n - \frac{v\Delta t}{2\Delta x}(u_{i+1}^n - u_{i-1}^n) + \frac{v^2\Delta t^2}{2\Delta x^2}(u_{i+1}^n - 2u_i^n + u_{i-1}^n)}$$

Example 6: Implementing and testing Lax-Wendroff method in python

- Revise the script to implement the numerical Lax-Wendroff scheme to solve 1D advection equation
- Plot the results
- Test the code stability by using different *dt* and/or *dx*

Check the corresponding section in:
Numerical_methods_Part_II_with_exercises.ipynb
Or
Numerical_methods_Part_II_with_exercises.py

One can show that the Lax–Wendroff method for advection equation is conditionally stable for $C \leq 1$.

Try to obtain the stability conditions!

Stability criteria - Von Neumann stability analysis

Goal: to investigate the propagation and amplification of numerical errors.

Any error $\epsilon(x,t)$ in a solution can be represented by a superposition of oscillating functions:

$$\epsilon(x,t) = \sum_{m=1}^{M} e^{\alpha_m t} e^{ik_m x}$$

For a given wavenumber, we have:

$$\epsilon_i^n = e^{\alpha t} e^{ikx}$$

$$\epsilon_i^{n+1} = e^{\alpha(t+\Delta t)} e^{ikx}$$

$$\epsilon_{i+1}^n = e^{\alpha t} e^{ik(x+\Delta x)}$$

$$\epsilon_{i+1}^{n+1} = e^{\alpha(t+\Delta t)} e^{ik(x+\Delta x)}$$

Definition: A numerical scheme is stable when the amplitude of error is damped. This is the case when the norm of the error <1:

$$|\epsilon(\alpha\Delta t)|^2 \leq 1$$

Stability analysis of diffusion forward method

$$u_i^{n+1} = u_i^n + F(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

$$e^{\alpha(t+\Delta t)} e^{ikx} = e^{\alpha t} e^{ikx} + F(e^{\alpha t} e^{ik(x+\Delta x)} - 2e^{\alpha t} e^{ikx} + e^{\alpha t} e^{ik(x-\Delta x)})$$

$$e^{\alpha(\Delta t)} = 1 + F(e^{ik\Delta x} - 2 + e^{-ik\Delta x})$$

$$e^{ik\Delta x} + e^{-ik\Delta x} = 2cos(k\Delta x)$$

$$e^{\alpha(\Delta t)} = 1 + 2F(cos(k\Delta x) - 1)$$

The condition that needs to be satisfied:

$$|\epsilon(\alpha\Delta t)|^2 \leq 1$$

$$(1 + 2F(cos(k\Delta x) - 1))^2 \leq 1$$

worst case for $cos(k\Delta x)$=−1:

$$(1 - 4F)^2 \leq 1$$

$$1 + 8F(2F - 1) \leq 1 \quad \Rightarrow \quad F \leq \frac{1}{2} \quad \Rightarrow \quad \boxed{\Delta t \leq \frac{\Delta x^2}{2D}}$$

$$8F(2F - 1) \leq 0$$

The forward method for diffusion equation is conditionally stable.

Stability analysis of diffusion backward method

$$-Fu_{i-1}^{n} + (1+2F)u_i^n - Fu_{i+1}^{n} = u_i^{n-1}$$

$$-Fe^{\alpha t}e^{ik(x-\Delta x)} + (1+2F)e^{\alpha t}e^{ikx} - Fe^{\alpha t}e^{ik(x+\Delta x)} = e^{\alpha(t-\Delta t)}e^{ikx}$$

$$e^{-\alpha(\Delta t)} = -Fe^{-ik\Delta x} - Fe^{ik\Delta x} + (1+2F)$$

Replacing and rearranging:

$$e^{ik\Delta x} + e^{-ik\Delta x} = 2cos(k\Delta x)$$

$$e^{-\alpha(\Delta t)} = (1+2F) - 2Fcos(k\Delta x)$$

$$e^{\alpha(\Delta t)} = \frac{1}{(1+2F) - 2Fcos(k\Delta x)}$$

We can show:

$$\boxed{|\epsilon(\alpha\Delta t)|^2 = \left(\frac{1}{(1+2F) + 2Fcos(k\Delta x)}\right)^2}$$

The backward method for diffusion equation is unconditionally stable.

Stability analysis of advection FTCS method

$$u_i^{n+1} = u_i^n - \frac{C}{2}(u_{i+1}^n - u_{i-1}^n)$$

$$e^{\alpha(\Delta t)} = 1 - \frac{C}{2}(e^{ik\Delta x} - e^{-ik\Delta x})$$

$$e^{\alpha(\Delta t)} = 1 - \frac{C}{2}2isin(k\Delta x)$$

By rearranging we can show:

$$\boxed{|\epsilon(\alpha\Delta t)|^2 = 1 + C^2 sin^2(k\Delta x) > 1}$$

The FTCS method for advection equation is unconditionally unstable.

Stability analysis of advection Lax–Friedrichs method

$$u_i^{n+1} = \frac{1}{2}(u_{i+1}^n + u_{i-1}^n) - \frac{v\Delta t}{2\Delta x}(u_{i+1}^n + u_{i-1}^n)$$

$$e^{\alpha(\Delta t)} = \frac{1}{2}\left(e^{ik\Delta x} + e^{-ik\Delta x}\right) - \frac{C}{2}\left(e^{ik\Delta x} - e^{-ik\Delta x}\right)$$

$$e^{\alpha(\Delta t)} = cos(k\Delta x) + iCsin(k\Delta x)$$

Replacing and rearranging:

$$|\epsilon(\alpha\Delta t)|^2 = cos^2(k\Delta x) - C^2 sin^2(k\Delta x)$$

$$|\epsilon(\alpha\Delta t)|^2 = 1 - sin^2(k\Delta x) - C^2 sin^2(k\Delta x)$$

$$|\epsilon(\alpha\Delta t)|^2 = 1 - (1 - C^2)sin^2(k\Delta x)$$

We can then show:

$$|\epsilon(\alpha\Delta t)|^2 \leq 1 \quad \Longrightarrow \quad 1 - (1 - C^2)sin^2(k\Delta x) \leq 1 \quad \Longrightarrow \quad \boxed{C \leq 1}$$

The Lax–Friedrichs method is conditionally stable for solving advection equation.

Numerical diffusion of advection Lax–Friedrichs method

$$u_i^{n+1} = \frac{1}{2}(u_{i+1}^n + u_{i-1}^n) - \frac{v\Delta t}{2\Delta x}(u_{i+1}^n - u_{i-1}^n)$$

$$u_i^{n+1} - u_i^n = \frac{1}{2}(u_{i+1}^n - 2u_i^n + u_{i-1}^n) - \frac{v\Delta t}{2\Delta x}(u_{i+1}^n - u_{i-1}^n)$$

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{1}{2\Delta t}(u_{i+1}^n - 2u_i^n + u_{i-1}^n) - v\frac{u_{i+1}^n - u_{i-1}^n}{2\Delta x}$$

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{(\Delta x)^2}{2\Delta t}\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} - v\frac{u_{i+1}^n - u_{i-1}^n}{\Delta x}$$

$$\frac{\partial u}{\partial t} = -v\frac{\partial u}{\partial x} + \boxed{\frac{(\Delta x)^2}{2\Delta t}\frac{\partial^2 u}{\partial x^2}}$$

Numerical diffusion

The Lax–Friedrichs method introduces *artificial (numerical) diffusion* to stabilize the advection equation.